

プログラムの設計方法 第二版(日本語訳) — Chapter 15 & 17 追加

Matthias Felleisen et al. (訳)

プログラムの設計方法 第二版

How to Design Programs, Second Edition

著者: Matthias Felleisen, Robert Bruce Findler, Matthew Flatt, Shriram Krishnamurthi

© 2014 (更新版 2026) MIT Press

この資料は著作権保護されており、Creative Commons [CC BY-NC-ND](https://creativecommons.org/licenses/by-nc-nd/4.0/) ライセンスの下で提供されています。

目次

前付け

- [序文 \(Preface\)](#)
 - 体系的なプログラム設計
 - DrRacket とティーチング言語
 - 移行可能なスキル
 - この本とその各部
 - 違い点
- [プロローグ: プログラムの仕方 \(Prologue: How to Program\)](#)
 - 算術と算術
 - 入力と出力
 - 計算のさまざまな方法
 - 1つのプログラム、多くの定義
 - もう1つの定義
 - あなたはもうプログラマだ
 - 違う!

本編

I 固定サイズのデータ (Fixed-Size Data)

- [1 算術](#)
 - 1.1 数の算術
 - 1.2 文字列の算術
 - 1.3 混ぜ合わせ

- 1.4 画像の算術
- 1.5 ブール値の算術
- 1.6 ブール値と混ぜ合わせ
- 1.7 述語: 自分のデータを知れ
- 2 関数とプログラム
 - 2.1 関数
 - 2.2 計算
 - 2.3 関数の合成
 - 2.4 グローバル定数
 - 2.5 プログラム
- 3 プログラムの設計方法 (How to Design Programs)
 - 3.1 関数の設計
 - 3.2 指の運動: 関数
 - 3.3 ドメイン知識
 - 3.4 関数からプログラムへ
 - 3.5 テストについて
 - 3.6 World プログラムの設計
 - 3.7 バーチャルペットの世界
- 4 区間、列挙、項目化 (Intervals, Enumerations, and Itemizations)
 - 4.1 条件式を使ったプログラミング
 - 4.2 条件付き計算
 - 4.3 列挙
 - 4.4 区間
 - 4.5 項目化
 - 4.6 項目化を使った設計
 - 4.7 有限状態の世界
- 5 構造の追加 (Adding Structure)
 - 5.1 位置から posn 構造へ
 - 5.2 posn を使った計算
 - 5.3 posn を使ったプログラミング
 - 5.4 構造型定義
 - 5.5 構造を使った計算
 - 5.6 構造を使ったプログラミング
 - 5.7 データの宇宙
 - 5.8 構造を使った設計
 - 5.9 World の中の構造
 - 5.10 グラフィカルエディタ
 - 5.11 さらに多くのバーチャルペット
- 6 項目化と構造 (Itemizations and Structures)
 - 6.1 再び、項目化を使った設計
 - 6.2 World を混ぜる
 - 6.3 入力エラー
 - 6.4 World の検査
 - 6.5 等価述語

- ・ [7 まとめ](#)

Intermezzo 1: Beginning Student Language

[BSL の語彙、文法、意味、エラーなど](#)

II 任意サイズのデータ (Arbitrarily Large Data)

- ・ [8 リスト \(Lists\)](#)
- ・ [9 自己参照データ定義を使った設計](#)
- ・ [10 リストの続き](#)
- ・ [11 合成による設計 \(Design by Composition\)](#)
- ・ [12 プロジェクト: リスト](#)
 - 実世界データ: 辞書、iTunes
 - 単語ゲーム、ワーム、単純テトリス、完全宇宙戦争、有限状態機械
- ・ [13 まとめ](#)

Intermezzo 2: Quote, Unquote

III 抽象化 (Abstraction)

- ・ [14 どこにでも類似点がある](#)
- ・ [15 抽象化の設計](#)
- ・ [16 抽象化の使用](#)
- ・ [17 無名関数 \(Nameless Functions / lambda\)](#)
- ・ [18 まとめ](#)

Intermezzo 3: Scope and Abstraction

IV 絡み合ったデータ (Intertwined Data)

- ・ [19 S 式の詩 \(The Poetry of S-expressions\)](#)
- ・ [20 反復的洗練 \(Iterative Refinement\)](#)
- ・ [21 インタプリタの洗練](#)
- ・ [22 プロジェクト: XML の商業](#)
- ・ [23 同時処理](#)
- ・ [24 まとめ](#)

Intermezzo 4: The Nature of Numbers

V 生成的再帰 (Generative Recursion)

- ・ [25 非標準の再帰](#)
- ・ [26 アルゴリズムの設計](#)
- ・ [27 バリエーション](#)
- ・ [28 数学的な例](#)
- ・ [29 バックトラックするアルゴリズム](#)
- ・ [30 まとめ](#)

Intermezzo 5: The Cost of Computation

VI 蓄積子 (Accumulators)

- ・ [31 知識の喪失](#)

- [32 蓄積子スタイル関数の設計](#)
- [33 蓄積のさらなる使用](#)
- [34 まとめ](#)

エピローグ: 先へ進む (Epilogue: Moving On)

注意: 本翻訳は学習・個人利用のためのものです。サンプルコードは原文のまま保持しています。

原典: <https://htdp.org/2026-5-28/Book/index.html> # 序文

Preface

	序文
	プロローグ: プログラムの仕方
I	固定サイズのデータ
	Intermezzo 1: Beginning Student Language
II	任意サイズのデータ
	Intermezzo 2: Quote, Unquote
III	抽象化
	Intermezzo 3: Scope and Abstraction
IV	絡み合ったデータ
	Intermezzo 4: The Nature of Numbers
V	生成的再帰
	Intermezzo 5: The Cost of Computation
VI	蓄積子
	エピローグ: 先へ進む

多くの職業で何らかのプログラミングが必要です。会計士はスプレッドシートをプログラムし、ミュージシャンはシンセサイザーをプログラムし、著者はワードプロセッサをプログラムし、Webデザイナーはスタイルシートをプログラムします。私たちが初版のこれらの言葉を書いたとき(1995–2000)、読者はそれらを未来的だと考えたかもしれません。今や、プログラミングは必須のスキルとなり、雇用見通しを高めるという目的で、数多くの書籍、オンラインコース、K-12 カリキュラムがこのニーズに応えています。

典型的なプログラミングのコースでは、「動くまでいじくり回す」アプローチを教えます。それが動くと、学生は「動いた!」と叫んで次に進みます。悲しいことに、このフレーズはコンピューティングにおける最短の嘘でもあり、多くの人々の人生から何時間も奪ってきました。対照的に、本書は良いプログラミングの習慣に焦点を当て、プロフェッショナルおよび職業プログラマの両方を対象としています。

「良いプログラミング」とは、ソフトウェア作成へのアプローチで、最初から、すべての段階で、すべてのステップで体系的な思考、計画、理解に依拠するものを意味します。この点を強調するために、私たちは「体系的なプログラム設計」と「体系的に設計されたプログラム」について語りまします。後者は、望まれる機能の根拠を明確に述べます。良いプログラミングは達成感の美的感覚も満たします。良いプログラムのエレガンスは、時の試練に耐えた詩や過去の時代の白黒写真に匹敵します。要するに、プログラミングと良いプログラミングの違いは、ダイナーのクレヨン画と

美術館の油絵のようなものです。

いいえ、この本は誰をもマスターペインターには変えません。しかし、私たちがこの版を書くのに 15 年を費やしたのは、以下を信じているからに他なりません。

誰もがプログラムを設計できる

そして

誰もが創造的設計から得られる満足感を体験できる

実際、私たちはさらに進んで主張します。

プログラム設計—— しかしプログラミングではない—— は、リベラルアーツ教育において数学や言語スキルと同じ役割を果たすべきである。

設計を学ぶ学生が二度とプログラムに触れなくても、普遍的に有用な問題解決スキルを身につけ、深く創造的な活動を経験し、新しい形式の美を理解することを学ぶでしょう。この序文の残りでは、「体系的設計」とは何を意味するのか、誰がどのように恩恵を受けるのか、そしてそれをどのように教えるのかを詳しく説明します。

体系的なプログラム設計

プログラムは人々（ユーザーと呼ばれる）と他のプログラム（サーバーとクライアントコンポーネントの場合）と相互作用します。したがって、合理的に完全なプログラムは多くの構成要素から成ります。一部は入力扱い、一部は出力を作成し、一部はそれらの間のギャップを橋渡しします。私たちは関数を基本的な構成要素として選ぶ理由は、誰もが前代数で関数に遭遇するからであり、最も単純なプログラムがまさにそのような関数だからです。鍵は、どの関数が必要か、それらをどのように接続するか、そして基本的な材料からそれらをどのように構築するかを発見することです。

この文脈で、「体系的なプログラム設計」とは、2 つの概念の混合を指します。**設計レシピ**と**反復的洗練**です。私たちは Michael Jackson の COBOL プログラム作成法、Daniel Friedman との再帰に関する会話、Robert Harper との型理論、Daniel Jackson とのソフトウェア設計から着想を得ました。設計レシピは著者らの創作であり、ここでは後者の使用を可能にします。

問題分析からデータ定義へ 表現しなければならない情報と、それが選択したプログラミング言語でどのように表現されるかを特定せよ。データ定義を定式化し、例で説明せよ。署名、目的文、ヘッダ 望まれる関数がどのような種類のデータを消費し、生成するかを述べよ。その関数が何を計算するのかという問いに対する簡潔な答えを定式化せよ。署名に沿ったスタブを定義せよ。関数の例 関数の目的を説明する例に取り組み。関数のテンプレート データ定義を関数のアウトラインに翻訳せよ。関数定義 関数のテンプレートの隙間を埋めよ。目的文と例を活用せよ。テスト 例をテストとして明確に述べ、関数がすべてに合格することを確認せよ。これにより誤りが発見される。テストはまた、必要が生じたときに他の人が定義を読み理解するのを助ける——そしてそれは深刻なプログラムでは必ず生じる。

図 1: 関数設計レシピの基本ステップ

設計レシピは完全なプログラムと個々の関数の両方に適用されます。本書は完全なプログラムのための 2 つのレシピだけを扱います。GUI を持つプログラム用とバッチプログラム用です。一

方、関数用の設計レシピにはさまざまなバリエーションがあります。数値のような原子的なデータ形式用、さまざまな種類のデータの列挙用、固定された方法で他のデータを複合するデータ用、有限だが任意に大きいデータ用、など。

関数レベルの設計レシピは共通の設計プロセスを共有します。図 1 はその 6 つの必須ステップを示しています。各ステップのタイトルは期待される成果を指定し、「コマンド」は主要な活動を示唆します。例はほぼすべての段階で中心的な役割を果たします。

各ステップには鋭い質問が付随しており、本書の 6 つのパートを通じて導入されます。特定のステップ—例えば関数の例の作成やテンプレート—では、質問はデータ定義に訴えかけるかもしれませんが。答えはほぼ自動的に中間成果物を作成します。

このアプローチの新しさは、初心者レベルのプログラムのための中間成果物の作成にあります。初心者が詰まったとき、専門家や講師は既存の中間成果物を検査できます。検査は設計プロセスの一般的な質問を使う可能性が高く、したがって初心者に自分で修正させることになります。そしてこの自己強化プロセスこそが、プログラミングとプログラム設計の重要な違いです。

反復的洗練は、問題が複雑で多面的であるという問題に対処します。一度にすべてを正しくすることはほぼ不可能です。代わりに、コンピュータ科学者はこの設計問題に取り組むために物理学から反復的洗練を借用します。本質的に、反復的洗練は、最初にすべての非本質的な詳細を取り除き、残された中核問題の解決策を見つけることを推奨します。洗練ステップは、これらの省略された詳細の 1 つを追加し、既存の解決策を可能な限り使用して拡張された問題を再解決します。これらの洗練ステップの繰り返し（反復とも呼ばれる）は、最終的に完全な解決策につながります。

この意味で、プログラマはミニ科学者です。科学者は世界の理想化されたバージョンのための近似モデルを作成し、それについての予測を行います。モデルの予測が当たっている限り、すべてはうまくいきます。予測された出来事が実際のもものと異なる場合、科学者はその不一致を減らすためにモデルを改訂します。同様に、プログラマにタスクが与えられたとき、彼らは最初の設計を作成し、それをコードにし、実際のユーザーで評価し、プログラムの振る舞いが望まれる製品に密接に一致するまで設計を反復的に洗練します。

本書は 2 つの異なる方法で反復的洗練を導入します。洗練による設計がプログラムの設計が複雑になると有用になるため、本書は問題がある程度の難易度を獲得した第 4 部でこの技法を明示的に導入します。さらに、私たちは第 1 部から第 3 部を通じて同じ問題のますます複雑なバリエーションを述べるために反復的洗練を使用します。つまり、中核問題を選び、1 つの章でそれを扱い、その後、後続の章で新たに導入された概念に合致する詳細で類似の問題を提示します。

DrRacket とティーチング言語

プログラムを設計することを学ぶには、繰り返しのハンズオン練習が必要です。ピアノを弾かずにピアノ奏者になれないのと同じように、実際のプログラムを作成して正しく動作させることなしにプログラム設計者にはなりません。したがって、本書には最小限のソフトウェアサポートが付属しています。プログラムを書き留めるための言語と、プログラムをワード文書のように編集し、プログラムを実行できるプログラム開発環境です。

多くの人が「コードの書き方を学びたい」と言い、次にどのプログラミング言語を学ぶべきかを尋ねます。いくつかのプログラミング言語が得ている注目を考えると、この質問は驚くべきことではありません。しかし、それは全く不適切でもあります。初心者を対象としないコースでは、既製の言語を設計レシピとともに使用できるかもしれません。現在流行のプログラミング言語でプロ

プログラミングを学ぶことは、学生を最終的な失敗に導くことがよくあります。この世界での流行は非常に短命です。「X で素早くプログラミング」本やコースは、次の流行の言語に移行できる原則を教えることに失敗することが多いです。さらに悪いことに、言語自体が、解決策を表現するレベルでもプログラミングミスに対処するレベルでも、移行可能なスキルの習得から気を逸らします。

対照的に、プログラムを設計することを学ぶことは、主に原則の研究と移行可能なスキルの習得についてです。理想的なプログラミング言語はこれら2つの目標をサポートしなければなりませんが、既製の産業用言語はそうしません。重要な問題は、初心者が言語の多くを知る前に間違いを犯すことです。しかしプログラミング言語は、プログラマがすでに言語全体を知っているかのように常にこれらのエラーを診断します。その結果、診断レポートは初心者をしばしば困惑させます。

私たちの解決策は、私たち自身の仕立てられたティーチング言語から始めることです。「Beginning Student Language」またはBSLと呼ばれます。この言語は本質的に、学生が前代数のコースで習得する「外国語」です。関数定義、関数適用、条件式のための記法を含みます。また、式を入れ子にすることもできます。この言語は非常に小さいので、言語全体の観点からのエラー診断でも、前代数しか持たない読者にとってまだアクセス可能です。

構造的設計原則を習得した学生は、次に「Intermediate Student Language」と他の高度な方言（総称して*SL）に進むことができます。本書はこれらの方言を使用して抽象化と一般再帰の設計原則を教えます。私たちは、このような一連のティーチング言語を使用することが、幅広い専門プログラミング言語（JavaScript、Python、Ruby、Java など）でプログラムを作成するための優れた準備を読者に提供すると固く信じています。

注: ティーチング言語は Racket で実装されています。Racket はプログラミング言語を構築するためのプログラミング言語として私たちが構築したものです。Racket はラボから現実世界に逃れ出しました。それはプログラミング言語です。

（続き: DrRacket の使用、ティーチング言語の選択など詳細）

移行可能なスキル

（中略: 設計レシピから得られる普遍的な問題解決スキル、創造的活動としてのプログラミングの価値について）

この本とその各部

本書は6つの主要なパートと5つの Intermezzo に分かれています。各パートは設計レシピの特定の側面に焦点を当て、徐々に複雑さを増していきます。

- Part I: 固定サイズのデータ — 基本的な算術から設計レシピの導入まで。
- Part II: 任意サイズのデータ — リストと自己参照データ定義。
- Part III: 抽象化 — 類似性の発見と高階関数。
- Part IV: 絡み合ったデータ — 複雑なデータ定義の相互作用。
- Part V: 生成的再帰 — 構造的再帰を超えたアルゴリズム。
- Part VI: 蓄積子 — 効率的な再帰のための追加の知識の保持。

各 Intermezzo では、言語の形式的な側面（文法、意味、計算モデル）を扱います。

違い点

(初版との違いの説明:より明確な設計レシピ、プロジェクトの強化など)

謝辞 (Acknowledgments)

…(詳細は原典を参照。多くの貢献者への感謝)

本翻訳は原典の内容を忠実に日本語化したものです。サンプルコードは原文を維持しています。

原典 URL: https://htdp.org/2026-5-28/Book/part_preface.html # プロローグ: プログラムの仕方

Prologue: How to Program

(目次リンクは省略)

あなたが小さな子供だったとき、親はあなたに数を数え、指で簡単な計算をすることを教えました。「 $1 + 1$ は 2」;「 $1 + 2$ は 3」;など。そして「 $3 + 2$ は？」と尋ね、あなたは片手の指を数えました。彼らはプログラムし、あなたは計算しました。そしてある意味で、それが本当にプログラミングとコンピューティングのすべてです。

今度は役割を交代する時です。DrRacket を起動しましょう。そうすると DrRacket のウィンドウが表示されます。「Language」メニューから「Choose language」を選び、「How to Design Programs」の「Teaching Languages」から「Beginning Student」(BSL)を選択し、OK をクリックして DrRacket を設定します。

これで準備完了です。あなたがプログラムし、DrRacket ソフトウェアが子供の役割を果たします。

最も単純な計算から始めましょう。DrRacket の上部に次のように入力します:

(+ 1 1)

RUN をクリックすると、下部に 2 が表示されます。

(図: DrRacket の画面)

それほど単純なのがプログラミングです。DrRacket を子供のように質問し、DrRacket があなたのために計算します。一度に複数のリクエストを処理させることもできます:

(+ 2 2)

(* 3 3)

(- 4 2)

(/ 6 2)

RUN をクリックすると、期待される結果 4 9 2 3 が表示されます。

少しペースを落として用語を導入しましょう。

DrRacket の上半分は「定義領域 (definitions area)」と呼ばれます。この領域でプログラムを作成します。これを編集と呼びます。定義領域に単語を追加したり変更したりすると、左上に SAVE ボタンが表示されます。初めて SAVE をクリックすると、DrRacket はファイルを保存するた

めの名前を尋ねます。一度ファイルに関連付けられると、SAVE をクリックすると定義領域の内容が安全にファイルに保存されます。

プログラムは式 (expressions) で構成されます。数学で式に出会ったことがあるでしょう。今のところ、式は単なる数値か、左括弧「(」で始まり対応する右括弧「)」で終わるものです——DrRacket はその括弧の間の領域を強調表示します。

RUN をクリックすると、DrRacket は定義領域の式を評価し、その結果を対話領域 (interactions area) に表示します。その後、DrRacket はプロンプト (>) であなたのコマンドを待ちます。プロンプトが表示されたら、追加の式を入力でき、DrRacket は定義領域と同じようにそれを評価します。

このようなプログラミングとコンピューティングを DrRacket で簡単に探求できます。

算術と算術

プログラミングが数と算術だけだったら、数学と同じくらい退屈でしょう。幸い、プログラミングには数値以外にも多くのものがあります。テキスト、真偽値、画像などです。

BSL で知っておくべき最初のことは、テキストは二重引用符 (") で囲まれた任意のキーボード文字のシーケンスであるということです。これを文字列 (string) と呼びます。したがって、

```
"hello world"
```

は完全に正しい文字列であり、DrRacket がこの文字列を評価すると、対話領域で数値と同じようにそのままエコーバックします。

BSL には数の算術に加えて、文字列の算術もあります。以下に 2 つの例を示します：

```
(string-append "hello" "world")  
; "helloworld"
```

```
(string-append "hello " "world")  
; "hello world"
```

+ と同様に、string-append は演算子です。2 番目の文字列を最初の文字列の末尾に追加して新しい文字列を作ります。最初の例が示すように、2 つの文字列の間に何も挿入せずに文字通り結合します。空白、カンマ、何もありません。

(以下、画像の算術、ブール値、条件式、関数の定義などプロローグの続きが展開されます。)

多くの方法で計算する

(条件式の導入例)

1 つのプログラム、多くの定義

(define を使った定数と関数の導入)

あなたはもうプログラマだ

ここまで来たら、あなたはすでに小さなプログラムを書けるプログラマです。以降の章で設計レ

シビを学び、体系的にプログラムを構築していきます。

注意: サンプルコードは原文のままです。

原典: https://htdp.org/2026-5-28/Book/part_prologue.html # 3 プログラムの設計方法

本書の最初の数章では、プログラミングを学ぶには多くの概念をある程度習得する必要があることを示してきました。一方では、プログラミングには言語が必要です。つまり、計算したいことを伝えるための表記法です。プログラムを定式化するための言語は人工的な構成物ですが、プログラミング言語の習得は自然言語の習得といくつかの要素を共有しています。どちらも語彙、文法、そして「句」が何を意味するかを理解を伴います。

他方では、問題文からプログラムに至る方法を学ぶことが重要です。問題文の中で何に関連していて、何を無視できるかを判断する必要があります。プログラムが何を消費し、何を生成し、入力と出力をどのように関連付けるかを明らかにする必要があります。選択した言語とそのティーチパックが、プログラムが処理するデータに対する特定の基本演算を提供しているかどうかを知る(または調べる)必要があります。提供していない場合は、それらの演算を実装する補助関数を開発しなければならないかもしれません。最後に、プログラムができたなら、それが実際に意図した計算を行っているかどうかを確認しなければなりません。そして、これはあらゆる種類のエラーを明らかにする可能性があり、それらを理解して修正できなければなりません。

これらすべてはかなり複雑に聞こえ、なぜただ実験しながら適当にやって、結果がまあまあならそのままにしておけばいいのかと思うかもしれません。このプログラミングへのアプローチはしばしば「ガレージプログラミング」と呼ばれ、多くの場合に成功します。時にはスタートアップ企業の立ち上げの足がかりになることもあります。しかし、スタートアップはその「ガレージ努力」の結果を販売することはできません。なぜなら、元のプログラマとその友人しか使えないからです。

良いプログラムには、それが何をするか、どのような入力を期待するか、何を生成するかを説明する短い書き込みが付属しています。理想的には、それが実際に動作することをある程度保証するものも付きます。最良の場合、プログラムの問題文へのつながりが明白なので、問題文への小さな変更をプログラムへの小さな変更簡単に翻訳できます。ソフトウェアエンジニアはこれを「プログラミング製品」と呼びます。

このすべての追加作業が必要なのは、プログラマが自分自身のためにプログラムを作成するわけではないからです。プログラマは、他のプログラマが読むためにプログラムを書きます。そして時折、人々は仕事をするためにこれらのプログラムを実行します。

「他の」という言葉には、通常、若い頃の自分がプログラムの制作に込めた思考のすべてを思い出すことができない年上のバージョンのプログラマも含まれます。

ほとんどのプログラムは、協力し合う関数の大規模で複雑な集合であり、誰もが1日でこれらすべての関数を書くことはできません。プログラマはプロジェクトに参加し、コードを書き、プロジェクトを去ります。他の人がそのプログラムを引き継いで作業します。もう一つの困難は、プログラマのクライアントが、本当に解決してほしい問題について考えを変えがちだということです。彼らは通常、ほぼ正しく理解していますが、多くの場合、いくつかの詳細を間違えます。さらに悪いことに、プログラムのような複雑な論理的構成物は、ほとんど常に人間のエラーに悩まされます。要するに、プログラマは間違いを犯します。最終的に誰かがこれらのエラーを発見し、プログラマはそれらを修正しなければなりません。彼らは1ヶ月前、1年前、または20年前のプログラムを読み直して変更する必要があります。

Exercise 33. 「year 2000」問題を調べよ。

ここでは、ステップバイステップのプロセスと、問題データを中心にプログラムを組織する方法を

統合した**設計レシピ**を提示します。長い間空白の画面を見つめ続けるのが嫌いな読者にとって、この設計レシピは体系的な方法で進捗を達成する方法を提供します。他の人をプログラム設計に教える人にとって、レシピは初心者の困難を診断するための道具です。他の人々にとっては、このレシピは医学、ジャーナリズム、工学など他の分野にも適用できるものかもしれません。本物のプログラマになりたい人にとって、設計レシピは既存のプログラムを理解し作業するための方法も提供します——ただし、すべてのプログラマがこのような設計レシピのような方法を使ってプログラムを考え出すわけではありません。この章の残りは、設計レシピの世界への最初の一步を捧げます。以降の章とパートは、レシピを何らかの形で洗練・拡張していきます。

3.1 関数の設計

情報とデータ

プログラムの目的は、ある情報を消費し、新しい情報を生成する計算プロセスを記述することです。この意味で、プログラムは数学の教師が生徒に与える指示のようなものです。しかし生徒とは異なり、プログラムは数値以上のものを扱います。ナビゲーション情報を計算し、人の住所を調べ、スイッチを入れ、ビデオゲームの状態を検査します。これらすべての情報は現実世界の一部——しばしばプログラムの**ドメイン**と呼ばれる——から来ており、プログラムの計算結果はこのドメイン内のさらなる情報を表します。

情報は私たちの説明で中心的な役割を果たします。情報をプログラムのドメインに関する事実として考えてください。家具カタログを扱うプログラムにとって、「5 本脚のテーブル」や「2 メートル四方の正方形テーブル」は情報の断片です。ゲームプログラムは異なる種類のドメインを扱い、「5」はあるオブジェクトがキャンパスのある部分から別の部分へ移動する際の、クロックティックあたりのピクセル数を指すかもしれません。また、給与計算プログラムは「5 つの控除」を扱う可能性が高いでしょう。

プログラムが情報を処理するためには、それをプログラミング言語の何らかの形の**データ**に変換しなければなりません。そしてデータを処理し、完了したら結果のデータを再び情報に変換します。対話型プログラムはこれらのステップをさらに混ぜ合わせ、必要に応じて世界からより多くの情報を取得し、その間に情報を配信するかもしれません。

私たちは BSL と DrRacket を使用するので、情報からデータへの変換について心配する必要はありません。DrRacket の BSL では、関数をデータに直接適用して何を生成するかを観察できます。その結果、情報をデータに変換したりその逆を行う関数を書くという深刻なニワトリと卵の問題を避けられます。単純な種類の情報の場合、そのようなプログラム部品の設計は些細です。単純な情報以外のものについては、例えばパーズングを知る必要があり、それは直ちにプログラム設計における多くの専門知識を必要とします。

ソフトウェアエンジニアは、BSL と DrRacket がデータ処理を情報→データのパーズングやデータ→情報への変換から分離する方法を **model-view-controller (MVC)** というスローガンで表現します。実際、十分に設計されたソフトウェアシステムはこの分離を強制することが今や受け入れられた知恵となっています。入門書の多くは依然としてそれらを混同していますが。したがって、BSL と DrRacket で作業することで、プログラムの核心の設計に集中でき、そこに十分な経験を積んだ後、情報-データ変換部分の設計を学ぶことができます。

ここでは、データと情報の分離を示すために、2 つのプリインストール済みティーチパックを使用します：`2htdp/batch-io` と `2htdp/universe` です。この章から、バッチプログラムと対話型プログラムのための設計レシピを開発し、完全なプログラムがどのように設計されるかのアイデアを提供します。完全なプログラミング言語のティーチパックは完全なプログラムのためのより多く

の文脈を提供しており、設計レシピを適切に適応させる必要があることを心に留めておいてください。

図 15: 情報からデータへ、そして戻る

情報とデータの中心的な役割を考えると、プログラム設計はそれらの間のつながりから始めなければなりません。具体的には、私たちプログラマは、選択したプログラミング言語を使って関連する情報の断片をデータとしてどのように**表現**し、データをどのように**解釈**して情報とするかを決定しなければなりません。図 15 はこの考えを抽象的な図で説明しています。

この考えを具体的にするために、いくつかの例を見てみましょう。数値の形で情報を消費・生成するプログラムを設計しているとします。表現の選択は簡単ですが、解釈には 42 のような数値がドメインで何を表すかを説明する必要があります：

- 42 は画像のドメインでは上端からのピクセル数を表すかもしれない
- 42 はシミュレーションやゲームオブジェクトが移動するクロックティックあたりのピクセル数を表すかもしれない
- 42 は物理学のドメインでは華氏、摂氏、または絶対温度目盛りでの温度を意味するかもしれない
- 42 はプログラムのドメインが家具カタログの場合、あるテーブルのサイズを指定するかもしれない
- 42 は単に文字列内の文字数を数えるかもしれない

鍵は、数値を情報として数値をデータとして(そしてその逆へ)どのように移行するかを知ることです。

この知識はプログラムを読むすべての者にとって非常に重要なので、私たちはしばしばそれをコメントの形で書き留め、**データ定義**と呼びます。データ定義には 2 つの目的があります。第一に、意味のある単語を使ってデータのあるコレクション(クラス)に名前を付けます。第二に、そのクラスの要素をどのように作成し、任意のデータ片がそのクラスに属するかどうかをどのように判断するかを読者に伝えます。

例：

```
; Temperature は Number  
; 摂氏での温度を表す
```

このデータ定義は、任意の数値を温度として解釈できることを述べています。-400 も Temperature です。読者はデータ定義から、-400 が有効な Temperature であることを理解します。

あるデータ定義が与えられたとき、読者は「このクラスに属するデータ片の例を挙げよ」と尋ねられるべきです。Temperature の場合、-273、0、100 は良い例です。読者はまた「この特定のデータはどのクラスに属するか？」と尋ねられるべきです。42 は Number、String、Image、Boolean のいずれでもありません。なぜなら文字列は Temperature に属さないからです。そして、Temperature のサンプルを作れと言われたら、-400 のようなものを思いつくかもしれません。

可能な最低温度が約-273℃であることを知っている場合、この知識をデータ定義で表現できるかどうかを疑問に思うかもしれません。データ定義は実際にはクラスの英語による記述に過ぎないので、確かにここに示したよりもはるかに正確な方法で温度のクラスを定義できます。本書では、このようなデータ定義に様式化された英語を使用し、次の章で「-274 より大きい」などの制約を課すためのスタイルを導入します。

これまで、4つのデータクラスの名前に出会いました: `Number`、`String`、`Image`、`Boolean` です。この知識があれば、新しいデータ定義を定式化することは、既存のデータ形式に新しい名前を導入することに他なりません。例えば「temperature」を数値に与えることです。この限られた知識でも、私たちの設計プロセスの概要を説明するのに十分です。

設計プロセス

入力情報をデータとして表現する方法と、出力データを情報として解釈する方法を理解したら、個々の関数の設計は簡単なプロセスに従って進められます。

1. 情報をデータとしてどのように表現したいかを表現せよ。1行のコメントで十分です:
; センチメートルを表すために数値を使います。
2. プログラムの成功する設計に関連すると考えるデータクラスに対して、`Temperature` のようなデータ定義を定式化せよ。
3. 署名、目的文、関数ヘッダを書け。

関数署名 は、設計の読者にあなたの関数がいくつの入力を消費し、それらがどのクラスから来ているか、何種類のデータを生成するかを伝えるコメントです。以下は3つの例です:

```
; String -> String
; String -> Number
; Number String Image -> Image
```

目的文 は、関数が何をするかを1文で述べるコメントです。良い目的文は、関数を読まずに何をするかを誰でも理解できるようにします。

ヘッダ(スタブとも呼ばれる)は単純化された関数定義です。署名内の各入力クラスに対して1つの変数名を選び、関数の本体は出力クラスからの任意のデータ片にできます。

```
(define (f a-string) 0)
(define (g n) "a")
(define (h num str img) (empty-scene 100 100))
```

パラメータ名は、パラメータが表すデータの種類を反映すべきです。時には、パラメータの目的を示唆する名前を使いたいと思うかもしれません。

目的文を定式化するとき、パラメータ名を使って何が計算されるかを明確にすると役立つことがよくあります。

4. 関数の例を作成せよ。

各関数について、少なくとも2~3個の具体例を挙げ、入力と期待される出力を書きましよう。これを「指の運動」と呼びます。

```
; given: "hello", expect: "h"
; given: "world", expect: "w"
```

5. テンプレートを書け。

データ定義から関数のアウトラインを導き出します。条件分岐が必要な場合は `cond` の骨組みを、構造体が必要なら選択子を使った骨組みを書きます。

6. 関数定義を完成させよ。

テンプレートを埋め、目的文と例を使って実際の計算ロジックを記述します。

7. テストを行え。

例を `check-expect` に変換し、すべてが通ることを確認します。

図 1: 関数設計レシピの基本ステップ(前述の 6 ステップの要約図)

このプロセスは初心者が進捗を確実にし、エラーを早期に発見するのを助けます。

関数の例から設計へ

ここでは、文字列の最初の文字を抽出する簡単な関数 `string-first` を設計する例を示します。

(コード例は原文のまま)

```
; String -> String  
; 文字列 s の最初の文字を抽出する  
(define (string-first s) "a")
```

```
; 例:  
; given: "hello", expect: "h"  
; given: "world", expect: "w"  
; given: " ", expect: " "
```

```
(check-expect (string-first "hello") "h")  
(check-expect (string-first "world") "w")
```

(以降の章の例も同様に設計レシピを適用して進めます。)

3.2 指の運動: 関数

この節の最初のいくつかの練習問題は、前の「関数」節のものとほぼコピーです。ただし、後者で「define」という単語を使っているところを、ここでは「design」という単語を使っています。この違いが意味するのは、設計レシピに従ってこれらの関数を作成し、解答には関連するすべての部品を含めるべきだということです。

節のタイトルが示唆するように、これらはプロセスを身体化するための練習問題です。ステップが第二の天性になるまで、1 つも飛ばさないでください。飛ばすと簡単に避けられるエラーにつながります。プログラミングには複雑なエラーの余地がたくさん残されています。ばかばかしいエラーに時間を無駄にする必要はありません。

Exercise 34. 関数 `string-first` を設計せよ。文字列 `s` から最初の文字を抽出する。

Exercise 35. 関数 `string-last` を設計せよ。文字列 `s` から最後の文字を抽出する。

Exercise 36. 関数 `image-area` を設計せよ。画像 `img` の面積を計算する。

Exercise 37. 関数 `string-rest` を設計せよ。文字列 `s` から最初の文字を除いた残りを返す。

Exercise 38. 関数 `string-remove-last` を設計せよ。文字列 `s` から最後の文字を除いた残りを返す。

(本書ではさらに多くの練習問題が続き、すべて設計レシピの適用を練習するものです。)

3.3 ドメイン知識

関数の本体をコード化するのにどのような知識が必要かを不思議に思うのは自然です。少し反省すると、このステップにはプログラムのドメインに対する適切な把握が必要であることがわかります。実際、そのようなドメイン知識には2つの形態があります：

1. 外部ドメインからの知識——数学、音楽、生物学、土木工学、芸術など。プログラマはコンピューティングのすべての応用ドメインを知ることはできないため、ドメイン専門家と問題を議論できるように、さまざまな応用分野の言語を理解する準備ができていなければなりません。数学は多くの(すべてではない)ドメインの交差点にあります。したがって、プログラマはドメイン専門家と問題を処理する際に新しい言語を習得しなければならないことがよくあります。
2. ティーチング言語とそのライブラリに関する知識。プログラマは、与えられた問題に対して言語が何を提供しているかを知る必要があります。

設計プロセスの中で、ドメイン知識は主にステップ5(関数定義を完成させる)で必要になりますが、良い例を作成するためにも役立ちます。

3.4 関数からプログラムへ

すべてのプログラムが単一の関数定義から成るわけではありません。いくつかの関数を必要とするものもあります。多くのものは定数定義も使用します。いずれにせよ、すべての関数を体系的に設計することは常に重要ですが、グローバル定数や補助関数は設計プロセスを少し変えます。

グローバル定数を定義した場合、関数は結果を計算するためにそれらを使用するかもしれませんが、それらの存在を自分に思い出させるために、テンプレートにこれらの定数を追加するとよいでしょう。結局のところ、それらは関数定義に寄与し得るものの目録に属します。

複数関数のプログラムは、対話型プログラムが自動的にキーイベントやマウスイベントを処理する関数、状態を画像として描画する関数などを必要とするために生じます。各補助関数も設計レシピに従って設計すべきです。

プログラム全体にはトップレベルの目的文も書くべきです。

3.5 テストについて

テストはすぐに労働集約的な作業になります。小さなプログラムを対話領域で実行するのは簡単ですが、多くの機械的な作業と複雑な検査が必要です。プログラマがシステムを成長させるにつれ、多くのテストを実施したいと思うようになります。やがてこの作業は圧倒的になり、プログラマはそれを怠るようになります。同時に、テストは基本的な欠陥を発見・防止するための最初の道具です。ずさんなテストはすぐにバグのある関数——隠れた問題を抱えた関数——につながり、バグのある関数はプロジェクトをしばしば複数の方法で遅らせます。

したがって、手動でテストを行うのではなく、テストを機械化することが重要です。多くのプログ

ラミング言語と同様に、BSL にはテスト機能が含まれており、DrRacket はこの機能を認識しています。

`check-expect` を使って例を自動テストに変換します：

```
(check-expect (string-first "hello") "h")
```

DrRacket は RUN 時にすべてのテストを実行し、結果を表示します。失敗したテストは赤で表示され、デバッグに役立ちます。

良いテストには以下が含まれます： - 典型的なケース - 境界値 (空文字列、長さ 1 の文字列など) - 特殊ケース

テストはまた、プログラムのドキュメントとしても機能します。

3.6 World プログラムの設計

前の章では `2htdp/universe` ティーチパックをアドホックな方法で紹介しましたが、この節では設計レシピがどのようにして world プログラムを体系的に作成するのを助けるかを示します。まず、データ定義と関数署名に基づく `2htdp/universe` ティーチパックの簡単な要約から始めます。次に、world プログラムのための設計レシピを詳述します。

ティーチパックは、プログラマが世界の状態を表すデータ定義と、すべての可能な世界状態に対して画像を作成する方法を知っている関数 `render` を開発することを期待します。プログラムの必要性に応じて、プログラマは次にクロックティック、キー入力、マウスイベントに応答する関数を設計しなければなりません。最後に、対話型プログラムは `big-bang` を使って状態、描画関数、イベントハンドラを結び付ける必要があります。

world プログラム設計レシピの概要：

1. 世界の状態を表すデータ定義を書く。
2. `render` 関数の署名と目的文を書く：`WorldState -> Image`
3. 必要に応じて `on-tick`、`on-key`、`on-mouse` などのハンドラの署名を書く。
4. 各関数について設計レシピを適用する。
5. `big-bang` でプログラムを組み立てる。

例として、シンプルなカウンターやアニメーションを設計できます。

3.7 バーチャルペットの世界

この練習問題の節は、バーチャルペットゲームの最初の 2 つの要素を紹介します。キャンバスを横切って歩き続ける猫の表示から始まります。もちろん、歩き続けると猫は不幸になり、その不幸が現れます。すべてのペットと同様に、撫でて少し助けるか、餌を与えてたくさん助けることができます。

まず、私たちのお気に入りの猫の画像から始めましょう：

```
(define cat1 <画像>)
```

猫の画像をコピーして DrRacket に貼り付け、`define` で画像に名前を付けましょう。

Exercise 45. 「バーチャルキャット」の world プログラムを設計せよ。猫を左から右に継続的に動

かす。`cat-prog` と呼び、猫の開始位置を消費すると仮定せよ。さらに、猫をクロックティックあたり 3 ピクセル動かせ。猫が右端で消えたら左端に再出現させる。`modulo` 関数について読むとよい。

Exercise 46. 少し異なる画像で猫のアニメーションを改善せよ:

```
(define cat2 <画像>)
```

Exercise 45 の描画関数を調整し、`x` 座標が奇数かどうかで 2 つの猫画像のどちらかを使うようにせよ。HelpDesk で `odd?` を調べ、`cond` 式を使って猫画像を選択せよ。

Exercise 47. 「幸福ゲージ」を維持・表示する `world` プログラムを設計せよ。`gauge-prog` と呼び、幸福度は 0 から 100 (両端を含む) の数値であると合意せよ。

各クロックティックで幸福レベルは 0.1 減少する。ただし 0 を下回らない。下矢印キーが押されるたびに幸福度は 1/5 減少、上矢印が押されると 1/3 ジャンプする。

幸福レベルを表示するために、黒い枠のある塗りつぶしの赤い長方形を持つシーンを使え。幸福レベル 0 では赤いバーは消え、最大幸福レベル `H` ではバーがシーン全体にわたるようにせよ。

Note. 十分な知識を得たら、`gauge-prog` を Exercise 45 の解と組み合わせる方法を説明する。その後、猫を助けることができるようになる。無視し続けると猫はますます不幸になる。撫でるとより幸せに、餌をやるとずっとずっと幸せになる。だからこそ、これらの最初の 3 章が教えてくれるよりもずっと多くの `world` プログラムの設計について知りたいと思う理由がわかるだろう。

注意: この章のすべての Racket コード、関数定義、`check-expect`、画像リテラル、`big-bang` 式などは原文のまま保持しています。翻訳は説明文、演習の記述、注記、設計プロセスの解説のみです。

原典: *How to Design Programs, Second Edition — Chapter 3 “How to Design Programs”* URL: [https://htdp.org/2026-5-28/Book/part_one.html# III](https://htdp.org/2026-5-28/Book/part_one.html#III) 抽象化 (Abstraction)

導入: 抽象化の必要性

多くのデータ定義と関数定義は似ています。例えば、`String` のリストの定義は `Number` のリストの定義と、データクラスの名前と「`String`」「`Number`」という単語の 2 箇所しか異なりません。同様に、`String` のリストから特定の文字列を探す関数は、`Number` のリストから特定の数を探す関数とほとんど区別がつきません。

経験から、このような類似性は問題を引き起こすことがわかっています。類似性は、プログラマがコードを (物理的または精神的に) コピーすることから生じます。プログラマが別の問題に直面したとき、既存の解決策をコピーして新しい問題に合わせて修正します。この行動は「本物の」プログラミングだけでなく、スプレッドシートや数学的モデリングの世界でも見られます。

しかし、コードのコピーは以下のような問題を引き起こします: - ミスのコピー - 同じ修正を多くのコピーに適用する必要性 - 基礎データ定義が変更されたときにすべてのコピーを探して修正するコスト

このプロセスは高価でエラーが発生しやすく、プログラミングチームに不必要なコストを課しま

す。

良いプログラマは、プログラミング言語が許す限り類似性を排除しようとします。

「プログラムはエッセイのようなものです。最初のバージョンは下書きであり、下書きには編集が必要です。」

「排除する」とは、プログラマがプログラムの最初のドラフトを書き留め、類似性（および他の問題）を見つけ、それらを取り除くことを意味します。最後のステップでは、**抽象化**を行うか、既存の（抽象化された）関数を使用します。このプロセスを何度か繰り返すことで、プログラムを満足のいく形にすることがよくあります。

このパートの前半では、関数とデータ定義の類似性を抽象化する方法を示します。プログラマはこのプロセスの結果も「抽象化」と呼びます。後半は、既存の抽象化の使用と、このプロセスを容易にする新しい言語要素についてです。このパートの例はリストの領域から取られています。考え方は普遍的に適用可能です。

14 類似性はどこにでもある (Similarities Everywhere)

Arbitrarily Large Data の演習（の一部）を解いた人は、多くの解決策が似ていることを知っているでしょう。実際、類似性は、ある問題の解決策をコピーして次の問題の解決策を作成する誘惑に駆られるかもしれません。

しかし、**thou shall not steal code**（コードを盗んではならない）、たとえ自分のコードであってもです。代わりに、類似したコード片を**抽象化**しなければなりません。この章では、その抽象化の方法を教えます。

私たちの手段は「Intermediate Student Language」（ISL）に特有のものです。

DrRacket で、「Language」メニューの「How to Design Programs」サブメニューから「Intermediate Student」を選択してください。

ほとんどすべての他のプログラミング言語も同様の手段を提供しています。オブジェクト指向言語では、追加の抽象化メカニズムが見つかるかもしれません。いずれにせよ、これらのメカニズムは本章で述べる基本的な特性を共有しており、ここで説明する設計の考え方は他の文脈でも適用されます。

14.1 関数における類似性 (Similarities in Functions)

設計レシピは、関数の基本的な組織を決定します。なぜなら、テンプレートは関数の目的に関係なくデータ定義から作成されるからです。したがって、同じ種類のデータを消費する関数は似ているのは当然です。

以下は 2 つの非常に似た関数です（図 86）：

```
; Los -> Boolean
; does l contain "dog"
(define (contains-dog? l)
  (cond
    [(empty? l) #false]
    [else
     (or
```

```

    (string=? (first l)
              "dog")
    (contains-dog?
      (rest l))))))
; Los -> Boolean
; does l contain "cat"
(define (contains-cat? l)
  (cond
    [(empty? l) #false]
    [else
     (or
      (string=? (first l)
                "cat")
      (contains-cat?
        (rest l))))]))

```

図 86: 2 つの類似した関数

これらの関数はほぼ区別が付きません。各々は文字列のリストを消費し、本体は 2 つの節を持つ cond 式で構成されます。入力が空リストなら #false を返し、or を使って最初の要素が目的の文字列かどうかを調べ、そうでなければ再帰的に残りを調べます。唯一の違いは比較する文字列です(影付きで強調)。

良いプログラマは、このような密接に関連する関数を複数定義するのを避けます。代わりに、探す文字列を追加の引数として受け取る 1 つの汎用関数を定義します:

```

; String Los -> Boolean
; determines whether l contains the string s
(define (contains? s l)
  (cond
    [(empty? l) #false]
    [else
     (or (string=? (first l) s)
         (contains? s (rest l))))]))

```

これで contains-dog? と contains-cat? は 1 行で定義できます:

```

; Los -> Boolean
; does l contain "dog"
(define (contains-dog? l)
  (contains? "dog" l))

; Los -> Boolean
; does l contain "cat"
(define (contains-cat? l)
  (contains? "cat" l))

```

図 87: 2 つの類似した関数、再考

これが関数的抽象化です。

Exercise 235. contains? を使って “atom”, “basic”, “zoo” を探す関数を定義せよ。

Exercise 236. 次の 2 つの関数のテストスイートを作成せよ:

```

; Lon -> Lon
; adds 1 to each item on l

```

```

(define (add1* l)
  (cond
    [(empty? l) '()]
    [else
     (cons
      (add1 (first l))
      (add1* (rest l)))]))
; Lon -> Lon
; adds 5 to each item on l
(define (plus5 l)
  (cond
    [(empty? l) '()]
    [else
     (cons
      (+ (first l) 5)
      (plus5 (rest l)))]))

```

これらを抽象化せよ。抽象化した関数を使って上記 2 つを 1 行で再定義し、テストで確認せよ。最後に、各数から 2 を引く関数を設計せよ。

(以下、14.2 Different Similarities、14.3 Similarities in Data Definitions、14.4 Functions Are Values、14.5 Computing with Functions の詳細な説明と例が続きます。)

14.2 異なる類似性

(本文の続き:異なるパターンの類似性についても同様に抽象化を適用。)

14.3 データ定義における類似性

Los と Lon のようなデータ定義の類似性も抽象化の対象です。パラメータ化されたデータ定義 `[List-of X]` を作成することで、汎用的に扱えます。

14.4 関数は値である

ISL では関数は値として扱えます。これにより関数を引数に渡す高階関数が可能になります。

14.5 関数を使った計算

(関数を値として扱う際の計算モデルと評価の説明。)

15 抽象化の設計 (Designing Abstractions)

15.1 例からの抽象化 (Abstractions from Examples)

足し算を初めて学んだとき、あなたは具体的な例を使って作業しました。親はおそらく指を使って小さな 2 つの数を足すことを教えたでしょう。後になって、任意の 2 つの数を足す方法を学びました。そこで初めてある種の抽象化に触れました。さらに後で、摂氏から華氏への温度変換や、与えられた速度と時間で車が移動する距離を計算する式を定式化することを学びました。

要するに、非常に具体的な例から抽象的な関係へと移行したのです。

```
; List-of-numbers -> List-of-numbers
; converts a list of Celsius
; temperatures to Fahrenheit
(define (cf* l)
  (cond
    [(empty? l) '()]
    [else
     (cons
      (C2F (first l))
      (cf* (rest l)))])])

; Inventory -> List-of-strings
; extracts the names of
; toys from an inventory
(define (names i)
  (cond
    [(empty? i) '()]
    [else
     (cons
      (IR-name (first i))
      (names (rest i)))])])

; Number -> Number
; converts one Celsius
; temperature to Fahrenheit
(define (C2F c)
  (+ (* 9/5 c) 32))
(define-struct IR
  [name price])
; An IR is a structure:
;   (make-IR String Number)
; An Inventory is one of:
; - '()
; - (cons IR Inventory)
```

図 90: 似た 2 つの関数

(類似した 2 つの関数 cf* と names を観察し、共通のパターンを見つける。)

2 つの関数はリストを処理し、各要素に関数を適用して cons で新しいリストを作っています。この共通性を抽出します。

```
(define (cf* l g)
  (cond
    [(empty? l) '()]
    [else
     (cons
      (g (first l))
      (cf* (rest l) g))])])
(define (names i g)
  (cond
    [(empty? i) '()]
    [else
     (cons
      (g (first i))
      (names (rest i) g))])])
```

```
(define (map1 k g)
  (cond
    [(empty? k) '()]
    [else
     (cons
      (g (first k))
      (map1 (rest k) g))]))
```

図 91: 同じ 2 つの似た関数を抽象化したもの

この節では、例から抽象化を作成するための設計レシピを紹介します。

15.2 署名における類似性 (Similarities in Signatures)

抽象化の過程で、関数の署名にも類似性が見られます。これにより、どの部分が共通でどの部分が異なるかを明確にできます。

15.3 単一の制御点 (Single Point of Control)

抽象化の利点の一つは、コードの変更箇所を 1 箇所に集中できることです。これによりメンテナンスが容易になります。

15.4 テンプレートからの抽象化 (Abstractions from Templates)

データ定義から導かれるテンプレートは、似たデータ構造に対する関数で共通のパターンを示します。このテンプレートを基に抽象化を設計します。

17 無名関数 (Nameless Functions)

17.1 lambda から生まれる関数 (Functions from lambda)

lambda の構文は次のとおりです：

```
(lambda (variable-1 ... variable-N) expression)
```

特徴はキーワード `lambda` です。続いて括弧で囲まれた変数列、そして結果を計算する式があります。

例 (1 引数)：

```
(lambda (x) (expt 10 x))
(lambda (x) (if (> x 0) x (- 0 x)))
(lambda (x) (string-append "hello " x))
```

これらは無名関数です。インタラクショナルエリアで直接使用できます。

17.2 lambda を使った計算 (Computing with lambda)

lambda 式は関数値として評価されます。

17.3 lambda を使った抽象化 (Abstracting with lambda)

lambda を使ってその場で関数を作成し、map や filter と組み合わせます。

```
(map (lambda (x) (* x x)) (list 1 2 3))
```

17.4 lambda を使った仕様 (Specifying with lambda)

テストケースや目的文で lambda を使って動作を指定できます。

17.5 lambda を使った表現 (Representing with lambda)

lambda は関数を値として表現するための強力な手段です。